

Instructions

6 exercises hain — error handling se lekar dynamic imports tak.

★ marked exercises interview mein often pooche jaate hain.

Code mentally trace karo pehle — phir Node.js mein run karke verify karo.

Exercise 1 ★ — Error Flow Predict Karo

Concept: `try/catch/finally` **Task:** Har case mein output predict karo — finally ke behavior pe dhyan do:

```
function test(x) {  
  try {  
    console.log('try start');  
    if (x === 1) throw new Error('Error at 1');  
    if (x === 2) return 'early return from try';  
    console.log('try end');  
    return 'try completed';  
  } catch (e) {  
    console.log('catch:', e.message);  
    return 'catch return';  
  } finally {  
    console.log('finally always runs');  
    // NOTE: return here would OVERRIDE try/catch return!  
  }  
}  
  
console.log(test(0)); // Q1: What prints?  
console.log('---');  
console.log(test(1)); // Q2: What prints?  
console.log('---');  
console.log(test(2)); // Q3: What prints?
```

Key Insight / Answer

test(0): 'try start' → 'try end' → 'finally always runs' → 'try completed'

test(1): 'try start' → 'catch: Error at 1' → 'finally always runs' → 'catch return'

test(2): 'try start' → 'finally always runs' → 'early return from try'

Key: finally HAMESHA run hota hai — even when return statement hit hota hai try mein.

Exercise 2 ★ — Custom Error Class Build Karo

Concept: Custom Errors **Task:** Form validation ke liye complete error hierarchy banao — HTTP status codes ke saath:

```
// Base application error
class AppError extends Error {
  constructor(message, code, statusCode = 500) {
    super(message);
    this.name = this.constructor.name; // auto class name!
    this.code = code;
    this.statusCode = statusCode;
    this.isOperational = true; // distinguishes from programming errors
  }
}

class ValidationError extends AppError {
  constructor(field, message) {
    super(message, 'VALIDATION_ERROR', 400);
    this.field = field;
  }
}

class AuthError extends AppError {
  constructor(message = 'Unauthorized') {
    super(message, 'AUTH_ERROR', 401);
  }
}

class NotFoundError extends AppError {
  constructor(resource) {
    super(`${resource} not found`, 'NOT_FOUND', 404);
    this.resource = resource;
  }
}

// Validate user registration
function validateRegistration({ name, email, password, age }) {
  if (!name || name.length < 2)
    throw new ValidationError('name', 'Name must be at least 2 characters');
  if (!email || !/^[@]+\.[^@]+\.$/.test(email))
    throw new ValidationError('email', 'Valid email required');
  if (!password || password.length < 8)
    throw new ValidationError('password', 'Password min 8 characters');
  if (age < 13)
    throw new ValidationError('age', 'Must be 13 or older');
  return true;
}

// Global error handler (Express-style)
function handleError(err, res) {
  if (err instanceof ValidationError) {
    return { status: err.statusCode, field: err.field, message: err.message };
  }
}
```

```
if (err instanceof AuthError) {
  return { status: 401, message: err.message };
}
if (err instanceof NotFoundError) {
  return { status: 404, resource: err.resource, message: err.message };
}
// Unknown error – don't expose details
return { status: 500, message: 'Internal Server Error' };
}
// Test
try { validateRegistration({ name: 'R', email: 'bad', password: '123', age: 20 }); }
catch (e) { console.log(handleError(e)); }
// { status: 400, field: 'name', message: 'Name must be at least 2 characters' }
```

Key Insight / Answer

this.name = this.constructor.name — automatically uses class name. No hardcoding!

isOperational: true — operational errors (expected) vs programming errors (bugs).

instanceof chain karo — specific se general order mein catch karo.

Unknown errors mein details expose mat karo — security risk hota hai.

Exercise 3 — Named vs Default Export/Import

Concept: ES Modules **Task:** Neeche diye code mein mistakes dhundo aur fix karo:

```
// config.js
const API_URL = 'https://api.example.com';
const TIMEOUT = 5000;
const MAX_RETRY = 3;
export default { API_URL, TIMEOUT }; // default export – object
export const MAX_RETRY = 3; // ERROR: re-declaration!

// Fix – separate named exports:
// config.js (FIXED)
export const API_URL = 'https://api.example.com';
export const TIMEOUT = 5000;
export const MAX_RETRY = 3;

// helpers.js
export function formatDate(date) {
  return new Date(date).toLocaleDateString('en-IN');
}
export function formatCurrency(amount) {
  return new Intl.NumberFormat('en-IN', {style: 'currency', currency: 'INR'}).format(amount);
}
export default class Logger {
  log(msg) { console.log(`[LOG] ${msg}`); }
}

// app.js – spot the import bugs
import { Logger } from './helpers.js'; // BUG 1: Logger is default export
import formatDate, formatCurrency from './helpers.js'; // BUG 2: missing braces
import * API_URL from './config.js'; // BUG 3: wrong syntax
// FIXED imports:
import Logger from './helpers.js'; // default – no braces
import { formatDate, formatCurrency } from './helpers.js'; // named – braces
import { API_URL, TIMEOUT } from './config.js'; // named – braces
import * as Config from './config.js'; // namespace – correct
```

Key Insight / Answer

Bug 1: Logger is default export — import Logger (no braces) nahi { Logger }.

Bug 2: Multiple named imports need braces: import { formatDate, formatCurrency }.

Bug 3: Namespace import syntax: import * as Config (not * API_URL).

Rule: default = no braces. Named = braces. Namespace = * as alias.

Exercise 4 — Barrel File (index.js) Pattern

Concept: [Module Organization](#) **Task:** Utils folder ke liye barrel pattern implement karo — consumers single import use karen:

```
// utils/  
// ■■■ string.js  
// ■■■ number.js  
// ■■■ date.js  
// ■■■ validators.js  
// ■■■ index.js <-- barrel file  
  
// utils/string.js  
export const capitalize = s => s.charAt(0).toUpperCase() + s.slice(1);  
export const slugify = s => s.toLowerCase().replace(/\s+/g, '-');  
export const truncate = (s, n) => s.length > n ? s.slice(0,n)+'...' : s;  
  
// utils/number.js  
export const clamp = (n, min, max) => Math.min(Math.max(n, min), max);  
export const formatINR = n =>  
  new Intl.NumberFormat('en-IN',{style:'currency',currency:'INR'}).format(n);  
  
// utils/validators.js  
export const isEmail = v => /^[^@]+@[^@]+\.[^@]+$/.test(v);  
export const isPhone = v => /^[6-9]\d{9}$/.test(v);  
export const isEmpty = v => v === null || v === undefined || v === '';  
  
// utils/index.js – barrel file (re-exports everything)  
export * from './string.js';  
export * from './number.js';  
export * from './validators.js';  
  
// ■ Consumer – single clean import!  
import { capitalize, slugify, formatINR, isEmail } from './utils/index.js';  
  
// Instead of:  
// import { capitalize } from './utils/string.js';  
// import { formatINR } from './utils/number.js';  
// import { isEmail } from './utils/validators.js';  
  
capitalize('hello world'); // 'Hello world'  
formatINR(50000); // '■50,000.00'  
isEmail('test@test.com'); // true
```

Key Insight / Answer

`export * from './file'` — sab named exports re-export karta hai.

Barrel files = single entry point. Consumers ko internal structure jaanna nahi.

Side effect: import side-effects bhi re-export hongi. Careful with heavy libraries.

Real world: React uses this — `import { useState, useEffect } from 'react'`.

Exercise 5 — Dynamic Import with Lazy Loading

Concept: `Dynamic import()` **Task:** User role ke basis pe alag modules dynamically load karo:

```
// Feature modules (pretend these exist)
// admin.js — AdminDashboard, manageUsers, viewLogs
// member.js — MemberDashboard, viewProfile
// analytics.js — charts, reports

// Dynamic role-based module loader
async function loadDashboard(role) {
  let module;
  try {
    switch(role) {
      case 'admin':
        module = await import('./admin.js');
        break;
      case 'member':
        module = await import('./member.js');
        break;
      default:
        throw new Error(`Unknown role: ${role}`);
    }
    return module.default; // default export access
  } catch (err) {
    console.error('Module load failed:', err.message);
    // Fallback to basic dashboard
    const { BasicDashboard } = await import('./basic.js');
    return BasicDashboard;
  }
}

// Load analytics only when user opens chart tab
document.getElementById('chart-tab').addEventListener('click', async () => {
  const { renderChart } = await import('./analytics.js');
  renderChart('#container', data);
  // analytics.js loads ONLY when chart tab is clicked!
});

// Preload on hover — UX trick
document.getElementById('chart-tab').addEventListener('mouseenter', () => {
  import('./analytics.js'); // preload without await — background load
});
```

Key Insight / Answer

`Dynamic import()` returns Promise — `await` karo ya `.then()` use karo.

`module.default` — dynamic import mein default export access karne ka tarika.

Error handling zaroori hai — module file exist na kare ya network error aaye.

Preload on hover — clever UX: user hover pe load shuru, click pe ready hota hai!

Concept: CJS vs ESM **Task:** CJS code ko ESM mein convert karo aur differences identify karo:

[illegible]

ESM mein file extensions (.js) required hain — CJS mein optional tha.

`module.exports = ...` → export / export default statements.

`require()` → import statements (static) ya `await import()` (dynamic).